



Hands-On Einführung in Game-Boy- Entwicklung

Universität Graz
Dipl.-Ing. Dr. Tristan Repolusk
22.06.2026

Was uns heute erwartet



00

Setup

- 09:00–09:10
- Link zu Repo & Folien
- Installation benötigter Software

02

Drei Regeln

- Wichtig zu beachten!
- Sonst evtl. Hardwareschäden

01

Intro

- 09:10–09:50
- Motivation: Warum Game Boy?
- Hardware & Komponenten
- Game-Boy-Assembly
- Entwicklungsumgebung
- Kurze Pause

03

Hands-On

- 10:00–11:15
- Kurze Pause
- 11:30–12:50

00

Setup

Kursmaterialien



- **Repository:**
<https://github.com/meteoritenhagel/kfu-gameboy-workshop/>
- **Slides (Deutsch):** gameboy-hands-on-DE.pdf
- **Slides (English):** gameboy-hands-on-EN.pdf



Installation



make (GPL-3.0 License): Build-Management

- Installation (<https://www.gnu.org/software/make/>)
 - Windows:
 - Download und installieren <https://www.msys2.org/#installation>
 - MSYS2 `pacman -S make`
 - MacOS: `brew install make`
 - Linux:
 - Apt: `sudo apt install make`
 - Arch: https://archlinux.org/packages/core/x86_64/make/
 - Fedora: <https://packages.fedoraproject.org/pkgs/make/make/>
- Testen: Kommando `make -v`

Sollte zurückgeben: z. B. `GNU Make 4.3`

Installation



git (GPL-2.0 License): Versionskontrolle

○ Installation (<https://git-scm.com/install/>)

- Windows: `MSYS2 pacman -S msys/git`
- MacOS: `brew install git`
- Linux:
 - Apt: `sudo apt install git`
 - Arch: https://archlinux.org/packages/extra/x86_64/git/
 - Fedora: <https://packages.fedoraproject.org/pkgs/git/git/>

○ Testen: Kommando `git -v`

Sollte zurückgeben: z. B. `git version 2.43.0`

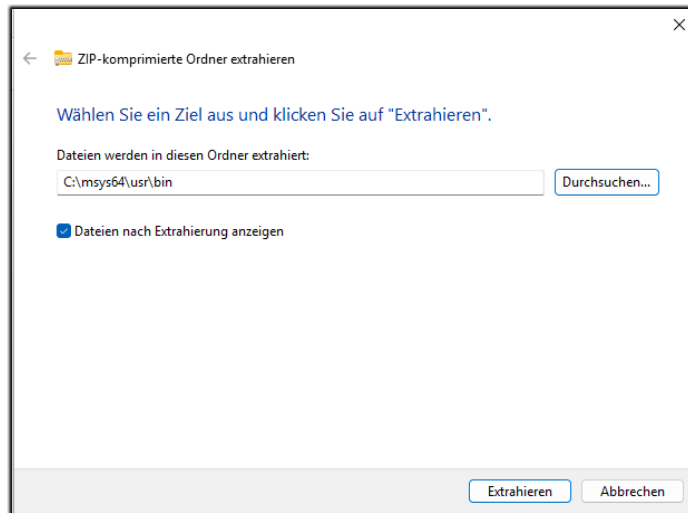
○ Repo klonen: `git clone https://github.com/meteoritenhagel/kfu-gameboy-workshop.git`

Installation



RGBDS (MIT License): Assembler/Linker

- Installation (<https://github.com/gbdev/rgbds>)
 - Richtiges Paket herunterladen:
github.com/gbdev/rgbds/releases/tag/v1.0.1/
 - Windows
 - Ins MSYS2-Verzeichnis `/usr/bin` extrahieren
(Standardwert: `C:\msys64\usr\bin`)
 - MacOS und Linux:
 - Extrahieren und `install.sh` ausführen
- Testen: Kommando `rgbasm -V`
- Sollte zurückgeben: `rgbasm v1.0.1`



Installation



Gearboy (GPL-3.0 License): Emulator/Debugger

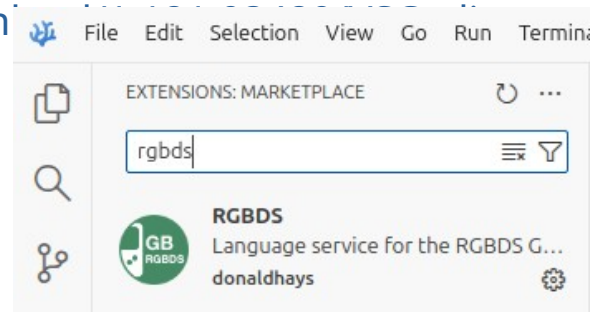
- Installation (<https://github.com/drhelius/Gearboy/>)
 - Richtiges Paket herunterladen und extrahieren:
github.com/drhelius/Gearboy/releases/tag/3.8.5/
 - Windows:
 - Ins MSYS2-Verzeichnis `/home` extrahieren
(Standardwert: `C:\msys64\home\<USERNAME>`)
 - `mv /usr/bin/Gearboy.exe /usr/bin/gearboy`
 - MacOS:
 - `ln -s <ABS_PATH>/Gearboy.app/Contents/MacOS/gearboy /usr/local/bin/gearboy`
 - Linux:
 - `ln -s <ABS_PATH>/gearboy /usr/local/bin/gearboy`
 - Testen: Kommando `gearboy --version`
Sollte zurückgeben: `Build: 3.8.5`

Installation



Text Editor, z. B. VSCodeium (MIT License)

- Installation (<https://vscodium.com/>)
 - Windows: Installation
<https://github.com/VSCodium/vscodium/releases/download/v1.121.03429/vscodium-user-setup-x64-1.121.03429.exe>
 - MacOS:
 - `brew install --cask vscodium`
 - Linux:
 - Snap: `snap install codium -classic`
 - Aura: `sudo aura -A vscodium-bin`
 - Flatpak: `flatpak install flathub com.vscodium.codium`
- RGBDS-Extension (MIT License): <https://github.com/DonaldHays/rgbds-vscode>
 - File > Preferences > Extensions



01

Intro

Warum Game Boy?



- Simple Architektur
- Gut dokumentiert
- Mit vielen Geräten abspielbar
- Hardware leicht erhältlich
- Nostalgie

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy (1989)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Super Game Boy (1994)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Pocket (1996)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Color (1998)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Advance (2001)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Player (2003)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Advance SP (2003)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

Hardware



System-on-a-Chip
Sharp LR35902
(CPU, PPU, APU)

4,19-MHz
Kristalloszillator

8 KB Video-RAM

8KB Working-RAM

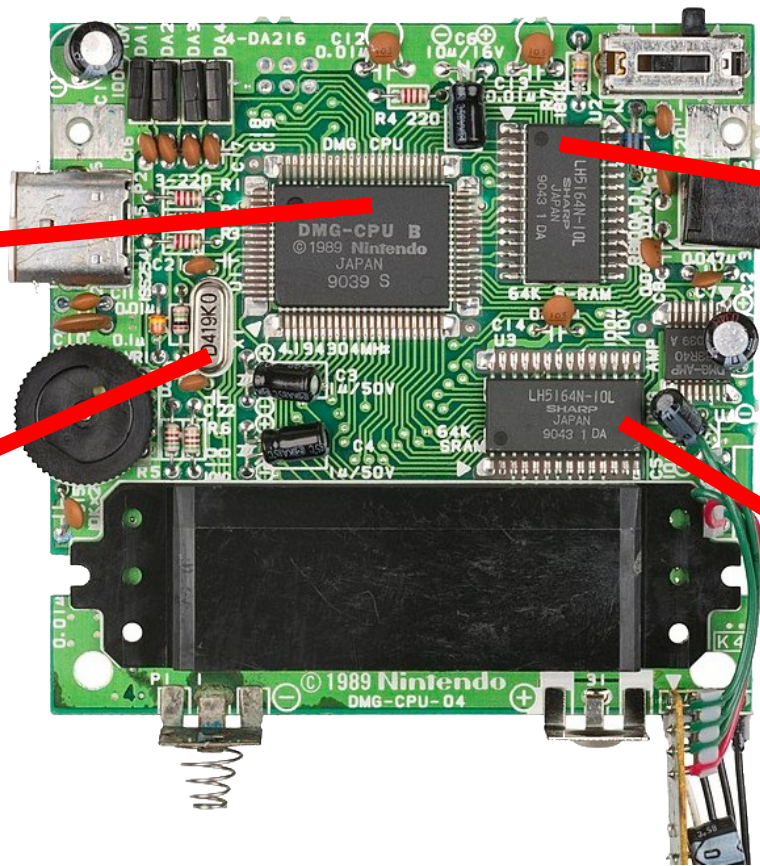


Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

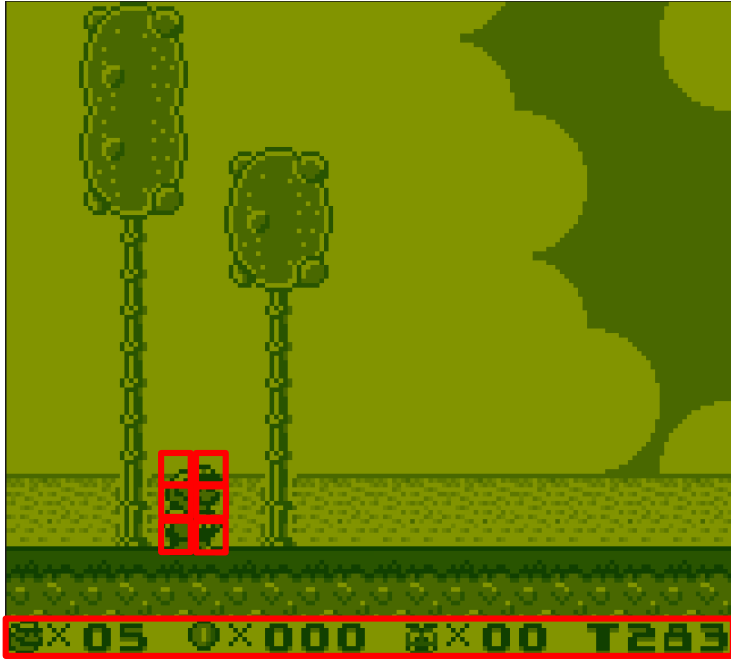
APU (Audio Processing Unit)



	 Duty 1	 Duty 2	 Wave	 Noise
0	D-4 03	D-4 05 CF2	D-4 06 C05	G#7 03
1				
2	D-4 03	A-4 05		
3				
4	A-4 03 0C0	D-4 05		G#7 03
5				
6		A-4 05		
7				
8	D-4 03	D-4 05		G#7 03

- Vier verschiedene Channel
(Square 1, Square 2, Wave, Noise)
- Kann direkt mit Befehlen von der CPU angesteuert werden (Memory Mapped I/O)
- Wir nutzen hier die Library hUGEDriver (Public Domain)
<https://github.com/SuperDisk/hUGEDriver>

PPU (Picture Processing Unit)



Quelle:
Super Mario Land 2. 6 Golden Coins [Nintendo Game Boy]. (1992).
Nintendo.

- Drei verschiedene Layer:
 - Background Layer (8x8-Tiles): Umgebung
 - Window Layer (8x8-Tiles, stark eingeschränkt): Statusleiste
 - Object Layer (8x8 bzw. 8x16 Pixel, pixelweise Position): Projektile
- Wir beschränken uns auf den Background Layer

PPU (Picture Processing Unit)



- 8 KB VRAM
- Bilder werden im 2bpp-Format gespeichert
 - 2 Bits pro Pixel → Werte 0, 1, 2 und 3
 - 1 Tile = 8x8 Pixel → 128 Bits = 16 Bytes



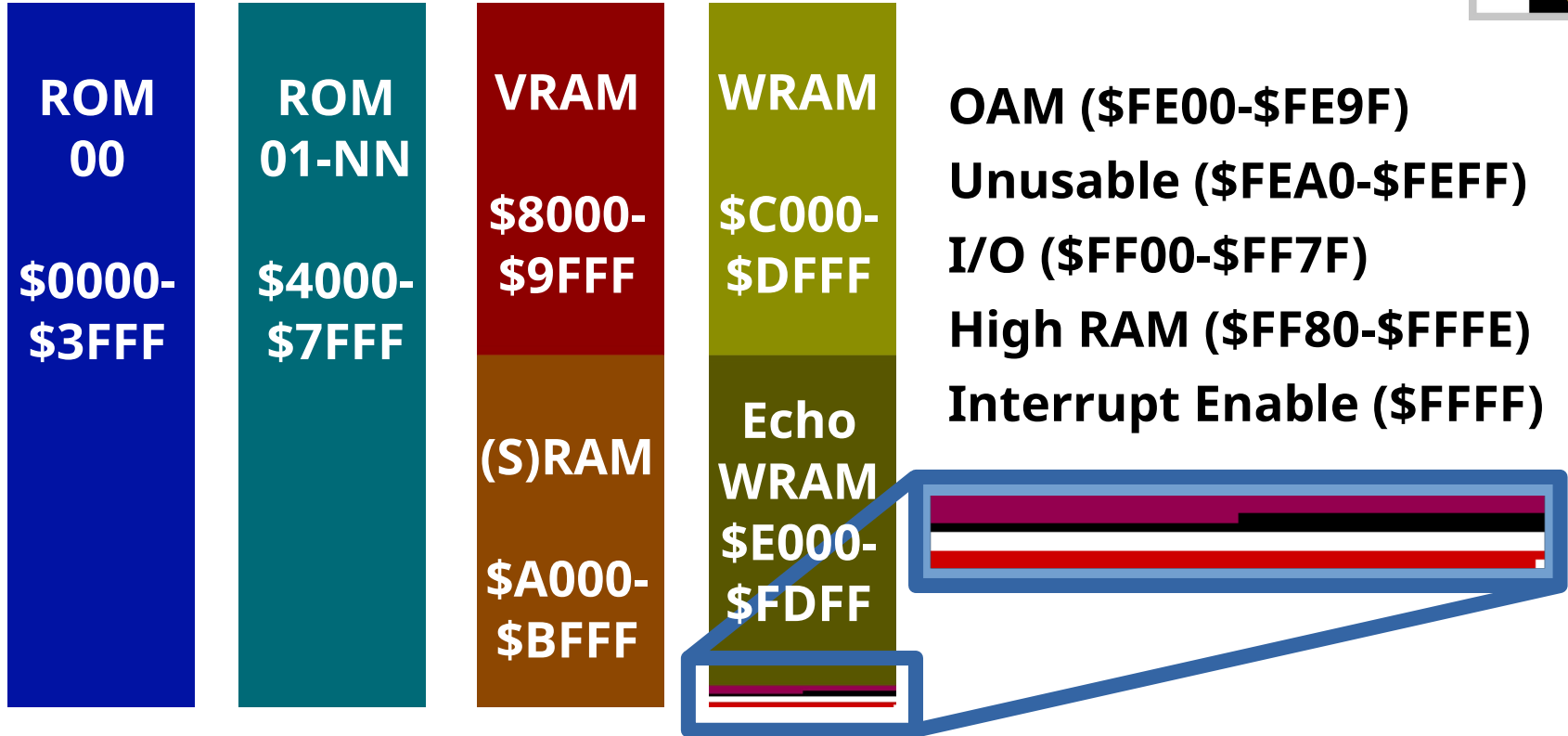
CPU (Central Processing Unit)

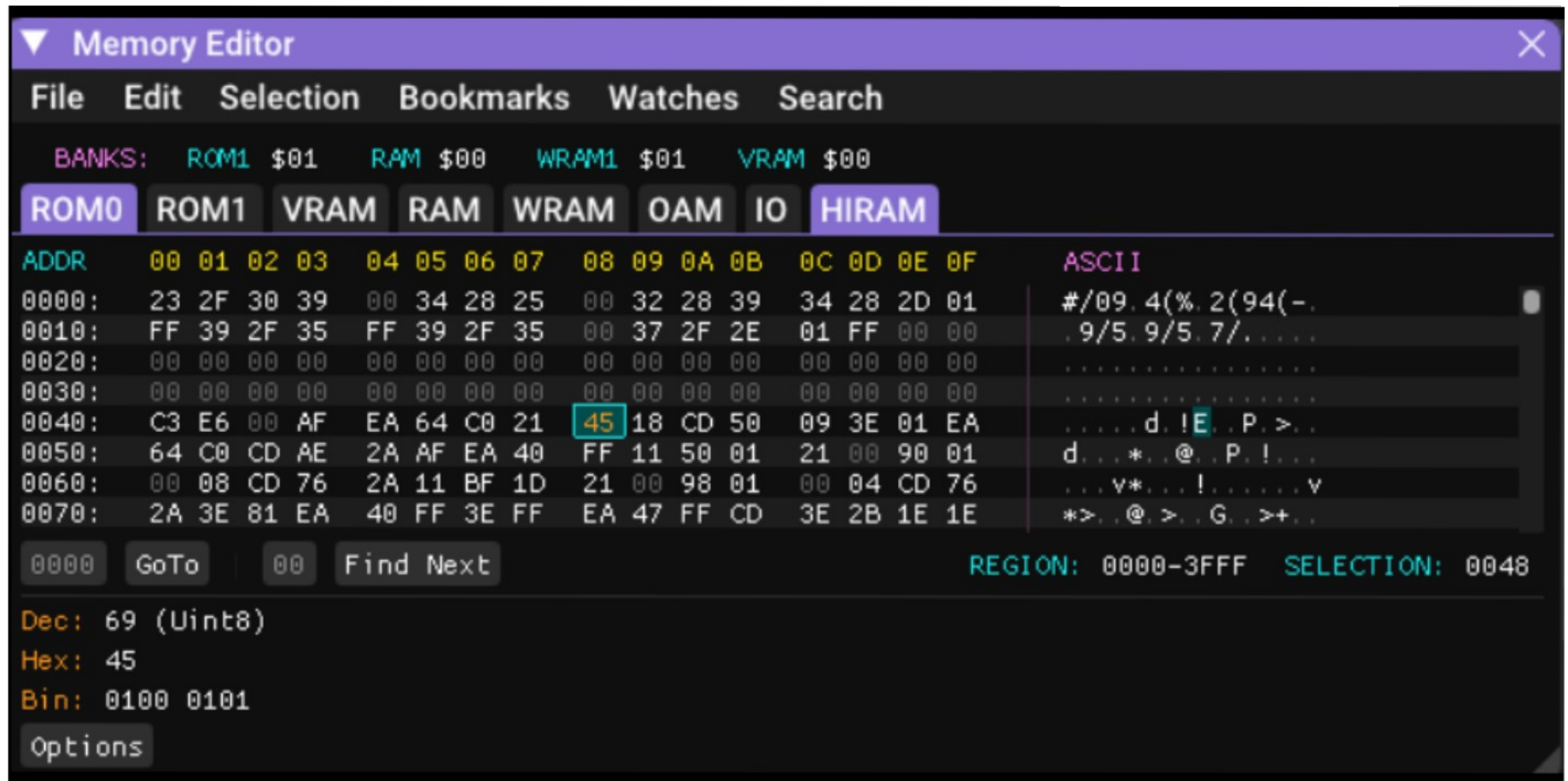


▼ Processor		✕	
Z = 0		N = 1	
H = 0		C = 1	
A = \$8F	F = \$50		
1000 1111	0101 0000		
B = \$00	C = \$00		
0000 0000	0000 0000		
D = \$21	E = \$1A		
0010 0001	0001 1010		
H = \$9C	L = \$00		
1001 1100	0000 0000		
SP = \$FFF8			
1111 1111 1111 1000			
PC = \$2AB6			
0010 1010 1011 0110			

- Sharp SM83
- 16-Bit Addressraum
- Elemente vom Intel 8080 und Zilog Z80
- Register: af (Accumulator + Flags), bc, de, hl
- Flags: z, n, h, c
(Zero, Subtraction, Half-Carry, Carry)
- Stack Pointer (SP) und Program Counter (PC)
- Memory Mapped I/O

Memory Map





Assembly



○ Maschinensprache:

- Befehl = Opcode + Daten,
- Beispiel: GB-Z80, lade Wert 255 in Register A:

`00111110 11111111` bzw. `0x3E 0xFF`

○ Assemblysprachen:

- 1:1-Beziehung zwischen Assemblycode und Maschinencode
- Befehl = Mnemonic + Daten
- Beispiel: RGBASM, lade Wert 255 in Register A:

`LD A, $FF`

Assembly



○ Höhere Programmiersprachen (z. B. C, C++, Python, ...):

- Höherer Abstraktionsgrad
- Leichter verständlich
- Beispiel (C):

Kopiere die ersten `n` Bytes von `source` nach `destination`

```
void memcpy(void *destination, const void *source, size_t n) {  
    uint8_t *dst = destination;  
    const uint8_t *src = source;  
    for (size_t i = 0; i < n; i++) {  
        dst[i] = src[i];  
    }  
}
```

C

```

void memcpy(void *destination, const void *source, size_t n) {
    uint8_t *dst = destination;
    const uint8_t *src = source;
    for (size_t i = 0; i < n; i++) {
        dst[i] = src[i];
    }
}

```



x86-64
ASM

```

; rsi: source
; rdi: destination
; rdx: n
memcpy:
    test rdx, rdx
    je .L1
    mov eax, 0

.L3:
    movzx ecx, BYTE PTR [rsi+rax]
    mov BYTE PTR [rdi+rax], cl
    add rax, 1
    cmp rdx, rax
    jne .L3

.L1:
    ret

```

GB-Z80
RGBASM

```

; de: source
; hl: destination
; bc: n
; only call with bc > 0
; destroys a
memcpy::
    ld a, [de]
    ld [hli], a
    inc de
    dec bc
    ld a, b
    or a, c
    jr nz, memcpy
    ret

```

Wichtigste Befehle



○ Ladeinstruktionen:

- `ld a, $64`: $a \leftarrow \$64$
- `ld a, b`: $a \leftarrow b$ (jedes andere Register möglich)
- `ld [$ABCD], a`: Lade a in das Byte mit Adresse \$ABCD
- `ld a, [$ABCD]`: Lade das Byte mit Adresse \$ABCD nach a.
- ...

○ Logische Operatoren immer mit dem Accumulator a

- `add a, b` (bzw. `add b`): $a \leftarrow a + b$
- statt `b` auch möglich: `a`, `c`, `d`, `e`, `h`, `l`, `[hl]`
- statt `and` andere Operationen: `or` oder `xor`

Wichtigste Befehle



- Kein If/While/For
- Dafür Sprunganweisungen:
 - Jump: `jp label`
 - Relative jump: `jr label`
 - Call subroutine: `call label`
 - Return from subroutine: `ret`, `reti`
- Bedingte Sprünge (Zero Flag, Carry Flag):
 - Jump if zero: `jp z, label`; jump if not zero: `jp nz, label`
 - Jump if carry: `jp c, label`; jump if not carry: `jp nc, label`
 - Äquivalent für `jp`, `call` und `ret`



Image: Public Domain by
<https://commons.wikimedia.org/wiki/User:Ocdp>

Kontrollstrukturen

Wir simulieren If-Statements:



CheckEqual::

```
ld a, [wVariable]
cp 5                ; if [wVariable] == 5
call z, VariableEqualFive ; call subroutine VariableEqualFive
ret
```

CheckLess::

```
ld a, [wVariable]
cp 5                ; if [wVariable] < 5
call c, VariableLessFive ; call subroutine VariableLessFive
ret
```

CheckGreaterOrEqual::

```
ld a, [wVariable]
cp 5                ; if [wVariable] >= 5
call nc, VariableGreaterOrEqualFive ; call subroutine GreaterOrEqualFive
ret
```

Kontrollstrukturen

Wir simulieren For-Statements:



```
ForLoop::  
    ld a, 10          ; iterate 10 times  
.loop  
    ; do something  
    dec a  
    jr nz, .loop  
    ret
```

Kontrollstrukturen

Wir simulieren Case-Statements:



```
CaseStatement::  
    ld a, [wVariable]  
    cp 0 ; in case [wVariable] == 0  
    jr nz, .notZero  
    call VariableIsZero ; call subroutine VariableIsZero  
    jr .done  
    .notZero  
    cp 1 ; in case [wVariable] == 1  
    jr nz, .notOne  
    call VariableIsOne ; call subroutine VariableIsOne  
    jr .done  
    .notOne  
    cp 2 ; in case [wVariable] == 2  
    jr nz, .done  
    call VariableIsTwo ; call subroutine VariableIsTwo  
    .done  
    ret ; or other exit, like jp, jr, ...  
; ...
```




Kurze Pause

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

02

Drei Regeln

Drei Regeln



1) Display ausschalten vor VRAM-Zugriffen



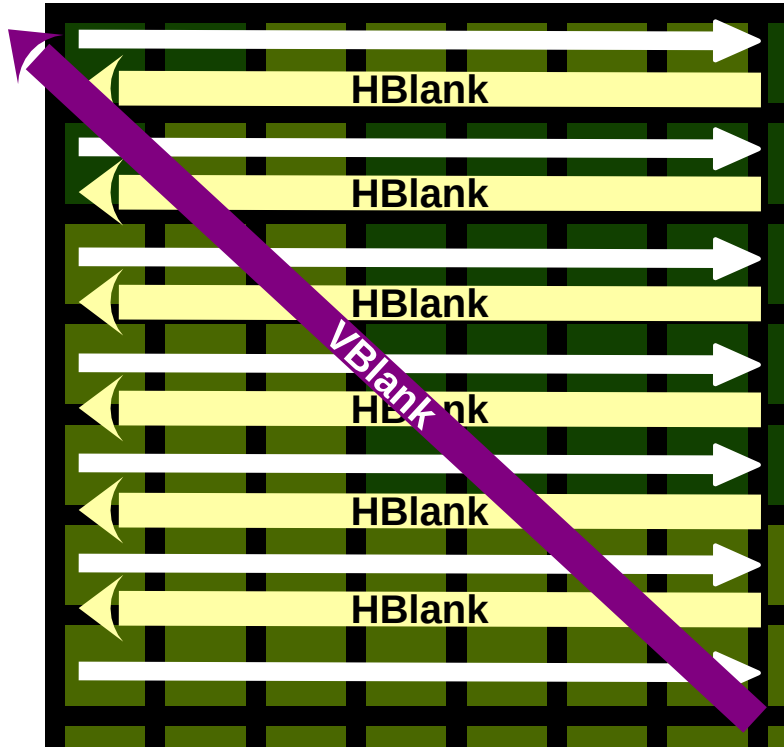
**2) Display nur während VBlank ausschalten!
(sonst evtl. Schäden am Display)**



3) Endlosschleifen statt Runaway-Code



VBlank und HBlank



- HBlank
 - Vom Ende einer Zeile zum Beginn der darunterliegenden springen
- VBlank
 - Vom Ende der letzten Zeile zurück zum Anfang springen
- Auflösung: 160x144 Pixel
 - 144 Zeilen zeichnen
 - 10 Zeilen VBlank (ca. 1.09 ms)

03

Hands-On

Empty Project

```
git checkout 00-empty-project
```



- main.asm
 - ROM Header
 - Entry Point
 - Busy Waiting
- utils.asm
 - Memcopy
 - UpdateKeys

Ü1: Grafiken anzeigen



Voraussetzung: `git checkout 00-empty-project`

Ziel: Den Title Screen anzeigen

- title.png von raw_resources nach project/res kopieren
- utils.asm: Schreibe die Funktion WaitVBlank
- Neues File state_title.asm anlegen
 - Schreibe InitStateTitle
 - WaitVBlank + LCD ausschalten
 - Tiles (\$9000) und Tilemap (\$9800) einbinden
 - Tiles kopieren, dann Tilemap kopieren
 - LCD einschalten und Palette initialisieren
- main.asm: InitStateTitle aufrufen



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/00-empty-project...01-title-screen-display>



Kurze Pause

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

Ü2: Text blinken lassen



Voraussetzung: `git checkout 01-title-screen-display`

Ziel: “Press Start to Play” zum Blinken bringen

- `utils.asm`: Schreibe die Funktion `WaitMultipleVBlank`, welche eine bestimmte Anzahl von VBlanks abwartet
- `state_title.asm`: Wechsle zwischen Paletten nach einer halben Sekunde, damit der Schriftzug “Press Start” blinkt



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/01-title-screen-display...02-title-screen-change-palette>

Ü3: Auf Eingabe reagieren



Voraussetzung:

`git checkout 02-title-screen-change-palette`

Ziel: Bei Tastendruck den Titelschirm verlassen

state_title.asm

- Neue Funktion WaitMultipleVBlankPressStart, welche jedes Frame auf Input überprüft
- Wenn Input erkannt wurde, verlassen wir den Title-Screen durch ret



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/02-title-screen-change-palette...03-title-screen-react-on-player-input>

Ü4: Musik hinzufügen



Voraussetzung:

```
git checkout 03-title-screen-react-on-player-input
```

Ziel: Ein Musikstück zum Klingen bringen

- Verschieben von raw_resources in das Projekt:
 - hUGEDriver/hUGE.inc → include
 - hUGEDriver/hUGE_note_table.inc → include
 - hUGEDriver/hUGEDriver.asm → src
 - huge_music/music_title.asm → src
- variables.asm: wUpdateSound
- start_title.asm: Laden des Musikstücks
- main.asm: Initialisieren + VBlank Interrupt Handler



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/03-title-screen-react-on-player-input..04-title-screen-add-music>



Flash Cart

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



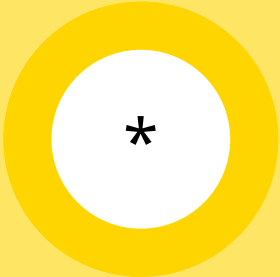
Demo am Emulator

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

Herzlichen Dank für Ihre Aufmerksamkeit.

Universität Graz
Institut für Digitale Geisteswissenschaften
Dipl.-Ing. Dr. Tristan Repolusk
Elisabethstraße 59/III, 8010 Graz, Österreich
tristan.repolusk@uni-graz.at
digital-humanities.uni-graz.at



A large yellow circle is positioned on the left side of the slide. Inside this circle is a smaller white circle, which contains a black asterisk (*).

*

Bonus

Ü5: Mehrere Game-States



Voraussetzung:

```
git checkout 04-title-screen-add-music
```

Ziel: Bei Tastendruck in den Game-State übergehen

- constants.inc: Füge zwei Konstanten für die beiden Game-States Title und Game hinzu
- variables.asm: wNextState
- Neues File: state_game.asm
 - Funktion InitStateGame
- main.asm: Statt .loop machen wir eine Entscheidung (wNextState) → entweder nach InitStateTitle oder InitStateGame
- title.asm: Vorm Return laden wir noch den nächsten geplanten State, also Game, nach wNextState



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/04-title-screen-add-music...05-switch-between-game-states>

Ü6: Grafiken vom Game-State



Voraussetzung:

`git checkout 05-switch-between-game-states`

Ziel: Die Grafik für die Spielmechanik anzeigen

- game.png von raw_resources nach res verschieben
- utils.asm: Neue Funktion StopSounds, welche alle vier Soundchannels stummschaltet
- state_game.asm:
 - Soundchannel stummschalten
 - Grafiken einbinden und anzeigen (vgl. state_title.asm)



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/05-switch-between-game-states...06-gameplay-display>

Ü7: Text anzeigen



Voraussetzung:

```
git checkout git checkout 06-gameplay-display
```

Ziel: Texte für die Spielmechanik anzeigen

- Verschieben von raw_resources nach project:
 - images/charmap.inc → include
 - images/font.png → res
- utils.asm: Neue Funktion DrawText, welche Text anzeigt
- state_game.asm:
 - Textdaten anlegen
 - Texttiles laden (\$8800)
 - Richtige Startposition für Text finden
 - DrawText aufrufen



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/06-gameplay-display...07-draw-text>

Ü8: Metronom (Teil 1)



Voraussetzung:

`git checkout 07-draw-text`

Ziel: Schlaggeräusch abspielen

- Kopieren `huge_music/music_weakbeat.asm` → `src`
- `utils.asm`:
 - Neue Funktion `PlayWeakBeat`
 - Soll schwachen Schlag abspielen (in `music_weakbeat.asm`)
 - Push alle Register
 - Laden des Sounds (siehe `state_title.asm`)
 - Pop alle Register



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/07-draw-text...08-gameplay-metronome>

Ü8: Metronom (Teil 2)



Ziel: Spielmechanik Metronom implementieren

- Startposition finden (rot markiert)
- Tile Number von Hintergrund/Pfeil finden?
- state_game.asm:
 - Lade Startposition nach hl
 - Funktion PlayMetronome schreiben:
 - PlayWeakBeat
 - Wechsle Grafik von Hintergrund auf Pfeil
 - Warte ein bisschen
 - Wechsle Grafik zurück auf leeres Feld
 - Insgesamt für alle acht Felder!



Ü9: Eingabe im Spiel (Teil 1)



Voraussetzung:

```
git checkout 08-gameplay-metronome
```

Ziel: Lauteres Schlaggeräusch abspielen

- huge_music/music_strongbeat.asm → ?
- utils.asm: PlayStrongBeat schreiben



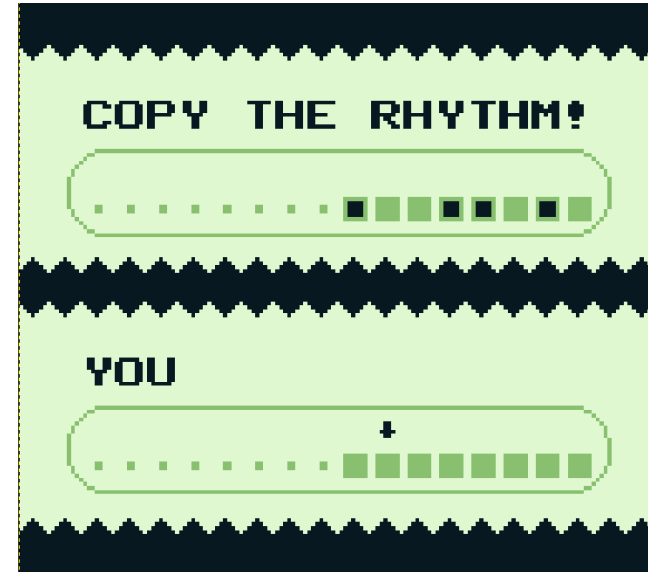
<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/08-gameplay-metronome...09-gameplay-react-on-player-input>

Ü9: Eingabe im Spiel (Teil 2)



Ziel: Spielereingabe → Box wird schwarz gefärbt

- Tile Number von leerer Box bzw. voller Box finden
- state_game.asm:
 - PlayerInput als Kopie von PlayMetronome anlegen
 - Statt WaitMultipleVBlank, neue Funktion WaitMultipleVBlankPlayerInput schreiben (vgl. state_title.asm)
 - Auf Playerinput prüfen:
 - falls ja: PlayStrongBeat und ändere Box auf voll (\$20 auf die Adresse addieren geht in nächste Zeile)
 - falls nein: Mach normal weiter



Weitere Übungen



- **10-gameplay-clear-boxes:**

Mehrere Gameplayrunden mit Zurücksetzen der Boxen

- **11-gameplay-given-beat-pattern:**

Pseudozufallszahlen als Startwert für den zu wiederholenden Beat

- **12-gameplay-store-player-input:**

Der Player-Input wird in eine Variable geschrieben

- **13-gameplay-compare-given-and-player-inputs:**

Wir vergleichen den gegebenen Beat mit dem Player-Input

Weitere Übungen



- **14-gameplay-change-tempo-each-round:**

Bei jeder erfolgreichen Runde wird das Tempo erhöht

- **15-gameplay-add-success-text:**

Ist das Spiel durchgespielt, erscheint ein Text "YOU WON!"

- **16-nice-screen-transitions:**

Jetzt noch ein bisschen die Übergänge zwischen Screens aufpolieren